

Question 1

- a) What is computer graphics? Explain two real-life uses of computer graphics.
- b) What is the difference between raster graphics and vector graphics? Give at least two examples of each.
- c) What are the features and advantages of interactive computer graphics? Give two examples of interactive graphics applications.
-

1(a) Define Computer Graphics. Explain two real-world applications.

Computer Graphics is the use of computers to create, edit, and display images, animations, and visual objects.

Applications

1. Computer-Aided Design (CAD)

- Used by engineers and architects.
- Helps design buildings, bridges, cars, and machines in 2D and 3D.

2. Entertainment

- Used in movies, video games, and animation.
- Creates realistic characters, visual effects, and animated scenes.

1(b) Differentiate between Raster Graphics and Vector Graphics

Raster Graphics	Vector Graphics
Made of pixels.	Made of lines and curves.
Lose quality when enlarged.	Can be enlarged without losing quality.
Best for photographs.	Best for logos and drawings.
Larger file size.	Smaller file size.

Examples

Raster Graphics

- JPEG
- PNG

Vector Graphics

- SVG
- AI (Adobe Illustrator)

1(c) Characteristics and Advantages of Interactive Computer Graphics

Characteristics

- User can interact with the graphics.
- Changes appear immediately.
- Easy to edit objects.
- Supports real-time input using mouse or keyboard.

Advantages

- Easy to use.
- Saves time.
- Gives instant feedback.
- Improves user experience.

Examples

- Video games
- CAD software

Question 2

a) Explain the Midpoint Circle Drawing Algorithm. Include:

- The basic idea of the algorithm
- The decision parameter and its initial value
- How the decision parameter is updated in each step
- Why only the points in the first octant need to be calculated

b) A circle has its center at (2, 1) and radius 8. Find the first 8 points in the first octant using the Midpoint Circle Drawing Algorithm.

2(a) Midpoint Circle Drawing Algorithm

Main Idea

The algorithm draws a circle by deciding which pixel is closest to the circle boundary using a decision parameter.

Decision Parameter and Initial Value

Initial decision parameter:

$$P_0 = 1 - r$$

where r is the radius.

Decision Parameter Update

If $P < 0$

$$P_{k+1} = P_k + 2x + 3$$

Choose **East (E)** pixel.

If $P \geq 0$

$$P_{k+1} = P_k + 2x - 2y + 5$$

Choose **South-East (SE)** pixel.

Why only first octant is calculated?

A circle is **8-way symmetric**. The points in the other seven octants are obtained by symmetry, so only the first octant needs to be calculated, making the algorithm faster.

2(b) First 8 Points of Circle

Step 1: Initial Values

- Center = (2,1)
- Radius = $r = 8$
- Initial point:

$$(x_0, y_0) = (0, 8)$$

- Initial decision parameter:

$$P_0 = 1 - r = 1 - 8 = -7$$

Step 2: Calculate the First 8 Points (First Octant)

k	P_k	Decision	(x_{k+1}, y_{k+1})	Actual Point $(x_c + x, y_c + y)$
0	-7	E	(1,8)	(3,9)
1	-4	E	(2,8)	(4,9)
2	1	SE	(3,7)	(5,8)
3	-4	E	(4,7)	(6,8)
4	5	SE	(5,6)	(7,7)
5	6	SE	(6,5)	(8,6)
6	11	SE	(7,4)	(9,5)

Including the starting point, the **first 8 points in the first octant** are:

Point No.	Relative Point (x,y)	Actual Coordinate
1	(0,8)	(2,9)
2	(1,8)	(3,9)
3	(2,8)	(4,9)
4	(3,7)	(5,8)
5	(4,7)	(6,8)
6	(5,6)	(7,7)
7	(6,5)	(8,6)
8	(7,4)	(9,5)

Answer (First 8 Points in the First Octant)

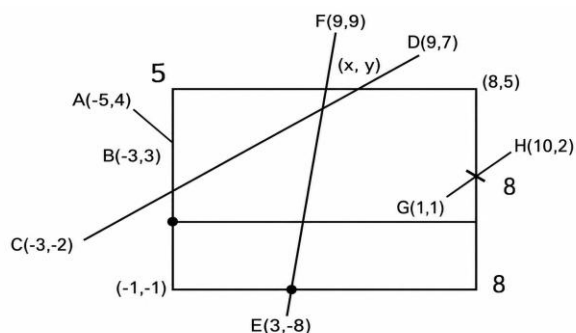
(2,9), (3,9), (4,9), (5,8), (6,8), (7,7), (8,6), (9,5)

Question 3

a) Explain the Cohen–Sutherland Line Clipping Algorithm. Include:

- Why line clipping is needed
- How region codes (outcodes) are created and used
- The three possible results after clipping a line

b) Use the Cohen–Sutherland Line Clipping Algorithm to clip the given lines. Find the intersection points and draw the clipped lines.

**3(a) Cohen-Sutherland Line Clipping Algorithm****Purpose**

It clips a line so that only the part inside the clipping window is displayed.

Region Codes

Each endpoint gets a **4-bit region code**.

Bits represent:

- Top
- Bottom
- Right
- Left

Example:

Inside = 0000

Three Possible Outcomes

1. Trivial Acceptance

- Both endpoints are inside.
- Draw the whole line.

2. Trivial Rejection

- Both endpoints are outside in the same region.
- Reject the line.

3. Partial Clipping

- One endpoint is inside and the other is outside.
- Find the intersection point and draw only the visible part.

3(b) Cohen-Sutherland Clipping Problem

Window:

- $X_{min} = -1$
- $X_{max} = 8$
- $Y_{min} = -1$
- $Y_{max} = 5$

Slope formula:

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

1) Line AB : A(-5,4) to B(-3,3)

Both endpoints are to the **left** of the window.

Result: Rejected (No intersection).

2) Line CD : C(-3,-2) to D(9,7)

Step 1: Find Slope

$$m = \frac{7 - (-2)}{9 - (-3)} = \frac{9}{12} = \frac{3}{4}$$

Step 2: Left Boundary ($x = X_{min} = -1$)

Using

$$\begin{aligned} y &= y_1 + m(x - x_1) \\ y &= -2 + \frac{3}{4}[-1 - (-3)] \\ &= -2 + \frac{3}{4}(2) = -2 + \frac{3}{2} = -\frac{1}{2} \end{aligned}$$

Intersection:

$$\boxed{(-1, -0.5)}$$

Step 3: Top Boundary ($y = Y_{max} = 5$)

Using

$$\begin{aligned}
 x &= x_1 + \frac{y - y_1}{m} \\
 x &= -3 + \frac{5 - (-2)}{\frac{3}{4}} \\
 &= -3 + \frac{7 \times 4}{3} = -3 + \frac{28}{3} = \frac{19}{3} \\
 &= \boxed{6.33}
 \end{aligned}$$

Intersection:

$$\boxed{(6.33, 5)}$$

Clipped Line

$$\boxed{(-1, -0.5) \rightarrow (6.33, 5)}$$

3) Line EF : E(3,-8) to F(9,9)**Step 1: Slope**

$$m = \frac{9 - (-8)}{9 - 3} = \frac{17}{6}$$

Step 2: Bottom Boundary ($y = -1$)

Using

$$\begin{aligned}
 x &= x_1 + \frac{y - y_1}{m} \\
 x &= 3 + \frac{-1 - (-8)}{\frac{17}{6}} \\
 &= 3 + \frac{7 \times 6}{17} = 3 + \frac{42}{17} = \frac{93}{17} \\
 &= \boxed{5.47}
 \end{aligned}$$

Intersection

$$\boxed{(5.47, -1)}$$

Step 3: Top Boundary ($y = 5$)

$$\begin{aligned}
 x &= 3 + \frac{5 - (-8)}{\frac{17}{6}} \\
 &= 3 + \frac{13 \times 6}{17} = 3 + \frac{78}{17} = \frac{129}{17} \\
 &= \boxed{7.59}
 \end{aligned}$$

Intersection

$$\boxed{(7.59, 5)}$$

Clipped Line

$$\boxed{(5.47, -1) \rightarrow (7.59, 5)}$$

4) Line GH : G(1,1) to H(10,2)

Step 1: Slope

$$m = \frac{2 - 1}{10 - 1} = \frac{1}{9}$$

Step 2: Right Boundary ($x = 8$)

Using

$$\begin{aligned} y &= y_1 + m(x - x_1) \\ y &= 1 + \frac{1}{9}(8 - 1) \\ &= 1 + \frac{7}{9} = \frac{16}{9} \\ &= \boxed{1.78} \end{aligned}$$

Intersection

$$\boxed{(8, 1.78)}$$

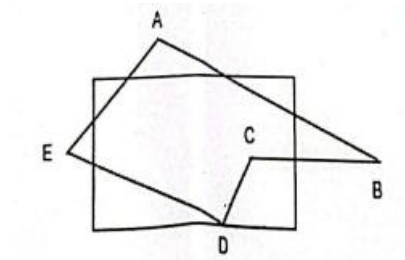
Clipped Line

$$\boxed{(1, 1) \rightarrow (8, 1.78)}$$

Question 4

a) Explain the Sutherland–Hodgman Polygon Clipping Algorithm. Describe the four cases that occur when a polygon edge is tested against a clipping boundary. Also state the output for each case.

b) Use the Sutherland–Hodgman Polygon Clipping Algorithm to clip the given polygon using the window defined by Xmax, Xmin, Ymax, and Ymin.



4(a) Sutherland-Hodgman Polygon Clipping Algorithm

There are **four cases**.

The 4 Cases (for each pair of adjacent vertices, checked against ONE boundary at a time)

Case	First Vertex	Second Vertex	Action (added to output list)
1	Outside	Inside	Add intersection point + second vertex
2	Inside	Inside	Add only the second vertex
3	Inside	Outside	Add only the intersection point
4	Outside	Outside	Add nothing

4(b) Sutherland-Hodgman Polygon Clipping

Since no vertex coordinates are given, the clipping is done **graphically** using the **Sutherland–Hodgman Polygon Clipping Algorithm**.

Clipping Order

Clip the polygon against the four window boundaries in this order:

1. **Left boundary** ($X_{\min}$)

2. **Right boundary** ($X_{\max}$)
3. **Bottom boundary** ($Y_{\min}$)
4. **Top boundary** ($Y_{\max}$)

The 4 Cases (for each pair of adjacent vertices, checked against ONE boundary at a time)

Case	First Vertex	Second Vertex	Action (added to output list)
1	Outside	Inside	Add intersection point + second vertex
2	Inside	Inside	Add only the second vertex
3	Inside	Outside	Add only the intersection point
4	Outside	Outside	Add nothing

Result for the Given Figure

After clipping:

- Vertex **A** is **outside above** the window, so it is replaced by **two intersection points** on the **top edge**.
- Vertex **B** is **outside to the right**, so it is replaced by **two intersection points** on the **right edge**.
- Vertex **E** is **outside to the left**, so it is replaced by **two intersection points** on the **left edge**.
- Vertex **C** lies **inside** the window and remains unchanged.
- Vertex **D** lies on the **bottom boundary**, so it is kept.

Question 5

a) Rotate the triangle A(0,0), B(1,1), and C(5,2) by 45° about the origin.

b) A 3D triangle has vertices A(4,5,1), B(6,4,2), and C(5,6,3). Reflect it about the ZX-plane and find the new coordinates.

5(a) Rotate Triangle by 45°

Given

A(0,0)

B(1,1)

C(5,2)

Rotation angle

$$\theta = 45^\circ$$

Rotation formulas

$$x' = x \cos \theta - y \sin \theta$$

$$y' = x \sin \theta + y \cos \theta$$

Since

$$\cos 45^\circ = \sin 45^\circ = 0.707$$

Point A(0,0)

$$x' = 0(0.7071) - 0(0.7071) = 0$$

$$y' = 0(0.7071) + 0(0.7071) = 0$$

$$A' = (0, 0)$$

Point B(1,1)

$$\begin{aligned}x' &= 1(0.7071) - 1(0.7071) = 0 \\y' &= 1(0.7071) + 1(0.7071) \\&= 0.7071 + 0.7071 = 1.4142\end{aligned}$$

$$B' = (0, 1.4142)$$

Point C(5,2)

$$\begin{aligned}x' &= 5(0.7071) - 2(0.7071) \\&= 3.5355 - 1.4142 = 2.1213 \\y' &= 5(0.7071) + 2(0.7071) \\&= 3.5355 + 1.4142 = 4.9497\end{aligned}$$

$$C' = (2.1213, 4.9497)$$

5(b) Reflection on ZX Plane

Reflection on the **ZX-plane** changes only the **y-coordinate**.

Rule:

$$(x, y, z) \rightarrow (x, -y, z)$$

Given

A(4,5,1)

B(6,4,2)

C(5,6,3)

After reflection

Reflection Rule (ZX-plane)

$$(x, y, z) \rightarrow (x, -y, z)$$

Step 1: Reflect Point A(4,5,1)

$$A(4,5,1) \rightarrow A'(4, -5, 1)$$

Step 2: Reflect Point B(6,4,2)

$$B(6,4,2) \rightarrow B'(6, -4, 2)$$

Step 3: Reflect Point C(5,6,3)

$$C(5,6,3) \rightarrow C'(5, -6, 3)$$

Question 6

a) Compare Perspective Projection and Parallel Projection with a neat diagram and proper explanation.

b) Write any three perspective projection problems (anomalies) that can happen in 3D computer graphics. Briefly explain each one.

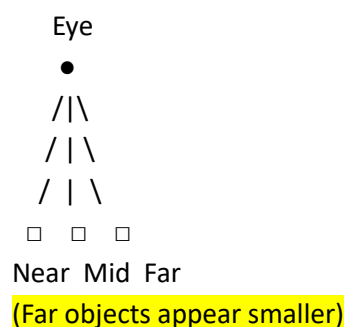
6(a) Compare Perspective Projection and Parallel Projection

Comparison Between Perspective Projection and Parallel Projection

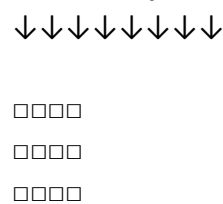
Perspective Projection	Parallel Projection
Projection lines meet at a single point (center of projection).	Projection lines remain parallel to each other.
Objects farther away appear smaller.	Objects keep the same size regardless of distance.
Produces a realistic view like human vision.	Produces a technical and accurate view.
Parallel lines may appear to converge (vanishing point).	Parallel lines remain parallel.
Distance and size are not preserved.	Distance and size are preserved (along parallel directions).
Used in games, movies, and realistic 3D graphics.	Used in engineering, architecture, and CAD drawings.
More realistic but mathematically more complex.	Simpler to implement and measure accurately.

Simple Diagram

Perspective Projection



Parallel Projection



(All objects keep the same size)

6(b) Three Perspective Anomalies

1. Foreshortening

Foreshortening is the effect where an object or part of an object appears shorter than its actual length when viewed at an angle. For example, a long road or railway track looks shorter as it extends into the distance. This happens because the depth of the object is compressed in perspective projection.

Cause: Perspective projection reduces the apparent length of objects as they move away from the viewer.

2. Size Reduction (Perspective Scaling)

In perspective projection, objects that are **farther from the viewer appear smaller**, while objects that are closer appear larger, even if they have the same actual size. For example, two identical buildings will appear to have different sizes if one is farther away.

Cause: The projected size of an object decreases as its distance from the eye (center of projection) increases.

3. Vanishing Point Effect

Parallel lines that never meet in the real world appear to converge at a single point in the distance, called the vanishing point. For example, railway tracks or the edges of a straight road seem to meet at the horizon.

Cause: In perspective projection, all projection lines pass through a single center of projection (the eye), causing parallel lines to appear as if they intersect.

Question 7

a) Convert these RGB colors into the CMYK color model:

- (1.0, 0.7, 0.5)
- (0.5, 0.4, 0.8)

Draw the RGB color cube.

b) In the given 2D figure:

- The eye is on the left looking along the line.
- A point light source is on the right.
- The light is twice as far from the line as the eye.
- The labeled points show the positions of the eye and the light.

i) If the line has only diffuse reflection, mark the point where it appears brightest to the viewer.

ii) If the line has only specular reflection, mark the point where it appears brightest to the viewer.

c) Explain the difference between Orthographic Projection and Oblique Projection.

7(a) Convert RGB to CMYK

Formula

$$K = 1 - \max(R, G, B)$$
$$C = \frac{1 - R - K}{1 - K}$$
$$M = \frac{1 - G - K}{1 - K}$$
$$Y = \frac{1 - B - K}{1 - K}$$

(1, 0, 0.7)

Largest value = 1

K = 0

C = 0

M = 1

Y = 0.3

Answer

CMYK = (0, 1, 0.3, 0)

(0.5, 0.4, 0.8)

Largest value = 0.8

$K = 1 - 0.8 = 0.2$

$C = (1 - 0.5 - 0.2) / 0.8 = 0.375$

$M = (1 - 0.4 - 0.2) / 0.8 = 0.5$

$Y = (1 - 0.8 - 0.2) / 0.8 = 0$

Answer

CMYK = (0.375, 0.5, 0, 0.2)

7(b) Diffuse and Specular Reflection

(i) Diffuse Reflection

Diffuse reflection depends only on the angle between the **light** and the **surface**. The brightest point is where the **light ray hits the plane most directly** (surface normal points toward the light).

Mark D: Near the point directly facing the light source.

(ii) Specular Reflection

Specular reflection depends on both the **light** and the **viewer (eye)**.

The brightest point is where the **reflected light goes directly toward the eye**.

Mark S: At the point where the reflection angle equals the viewing angle.

c) Explain the difference between Orthographic Projection and Oblique Projection.

Orthographic Projection	Oblique Projection
Projection lines are perpendicular (90°) to the projection plane.	Projection lines are inclined (not 90°) to the projection plane.
The front face is shown in its true size and shape .	The front face is shown in its true size , but the depth is drawn at an angle.
Parallel lines remain parallel after projection.	Parallel lines also remain parallel , but the depth direction is slanted.
There is no distortion of dimensions on the front face.	The depth may appear distorted or shortened .
Produces accurate measurements, so it is used for engineering and technical drawings .	Produces a simple 3D appearance, so it is used for illustrations and sketches .
Examples: Front view, Top view, Side view in CAD drawings.	Examples: Cabinet projection and Cavalier projection.

Question 8

a) What is a Bezier Curve? Write its properties. How are Hermite Geometry and Bezier Geometry related? Explain.

b) Explain the different ways of representing polygonal meshes.

c) Explain Gouraud Shading and Phong Shading. Also explain why the Gouraud Shading algorithm can be easily combined with the scan-line hidden surface algorithm.

8(a) What is a Bézier Curve?

Definition

A **Bézier Curve** is a smooth curve created using a set of **control points**. It is widely used in computer graphics for drawing smooth shapes.

Properties

- Smooth and continuous curve.
- Controlled by control points.
- Starts at the first control point.
- Ends at the last control point.
- Always lies inside the convex hull of its control points.

Relation Between Hermite and Bézier Curves

- Both are **parametric curves**.
- Both produce smooth curves.
- **Hermite curve** uses **endpoints and tangent vectors**.
- **Bézier curve** uses **control points**.
- Both can be converted into each other mathematically.

8(b) Different Ways of Representing Polygonal Meshes

1. Face-Vertex Representation

Stores vertices and the vertices that form each face.

2. Edge List Representation

Stores vertices and edges connecting them.

3. Winged-Edge (or Half-Edge) Representation

Stores edges with information about neighboring faces and vertices for efficient editing.

8(c) Gouraud Shading and Phong Shading

Gouraud Shading

- Lighting is calculated at the **vertices**.
- Colors are interpolated across the polygon.
- Fast but may miss shiny highlights.

Phong Shading

- Normals are interpolated across the polygon.
- Lighting is calculated for **every pixel**.
- Produces smoother and more realistic images.
- Slower than Gouraud shading.

Comparison

Gouraud Shading	Phong Shading
-----------------	---------------

Lighting at vertices	Lighting at every pixel
----------------------	-------------------------

Faster	Slower
--------	--------

Less realistic	More realistic
----------------	----------------

May miss highlights	Produces accurate highlights
---------------------	------------------------------